

Lecture Slide - 1
Accuracy, Precision &
Different Types of Errors

Accuracy vs Precision

- In chemistry, the meanings of accuracy and precision are quite different.
- **Accuracy** is a measure of how close a measurement comes to the actual or true value of whatever is measured.
- **Precision** is a measure of how close a series of measurements is to one another, irrespective of the actual value.

Accuracy vs Precision

- To evaluate the **accuracy** of a measurement, the measured value must be compared to the correct value. Or,
- **Accuracy** is related to the closeness to the correct/true value.

- To evaluate the **precision** of a measurement, you must compare the values of two or more repeated measurements. Or,
- **Precision** is related to the closeness to other estimated/measured values.

Accuracy vs Precision

Darts on a dartboard illustrate the difference between accuracy and precision.



**Good Accuracy,
Good Precision**



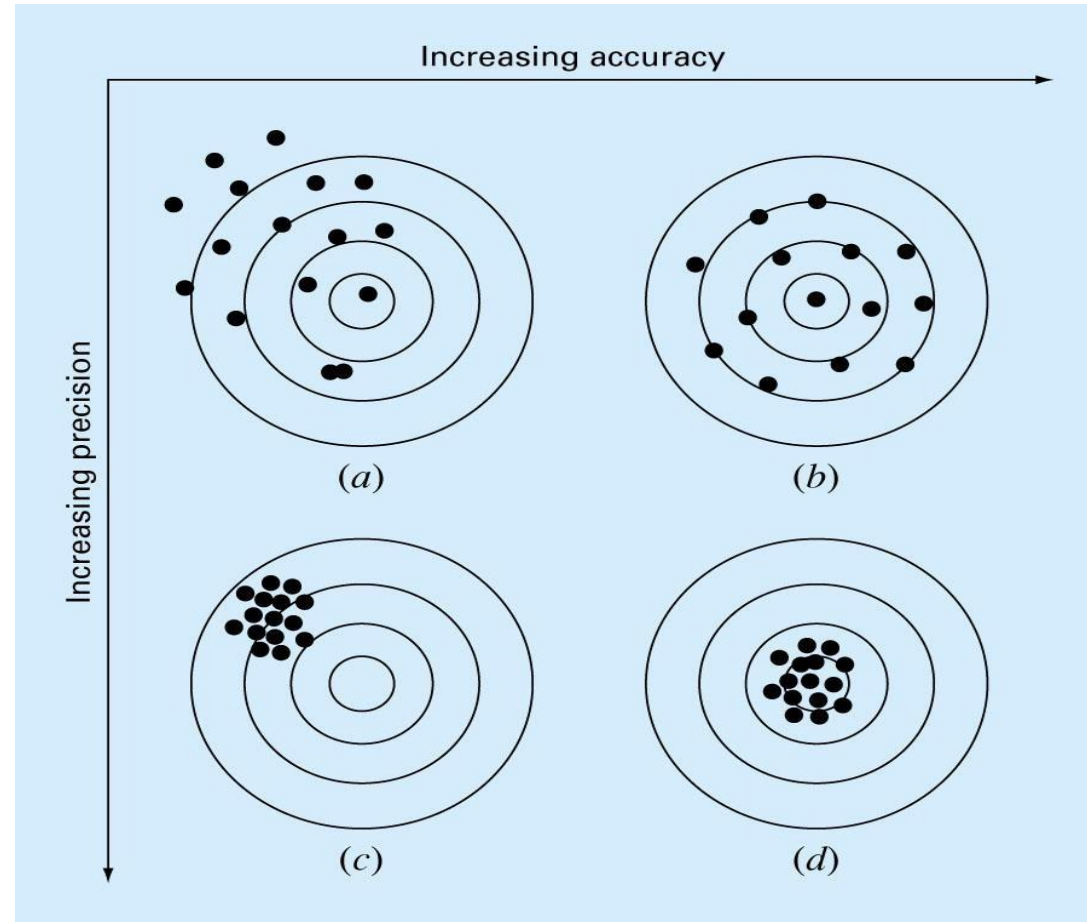
**Poor Accuracy,
Good Precision**



**Poor Accuracy,
Poor Precision**

The closeness of a dart to the bull's-eye corresponds to the degree of accuracy. The closeness of several darts to one another corresponds to the degree of precision.

Accuracy vs Precision



An example from marksmanship illustrating the concepts of accuracy and precision: (a) inaccurate and imprecise, (b) accurate and imprecise, (c) inaccurate and precise, and (d) accurate and precise.

Error

Determining Error

- There is a difference between the accepted value, which is the correct value for the measurement based on reliable references, and the experimental value, the value measured in the lab.
- The difference between the experimental value and the accepted value is called the error.

Error

Definitions

- Numerical errors arise from **the use of approximations** to represent exact mathematical operations and quantities.
- For such errors, the relationship between the exact, or true, result and the approximation can be formulated as
- **True value – Approximation = Error**
- **ε_t = Accepted value – Experimental value**
- ε_t is used to designate the exact value of the error

Error

Determining Error

Note that the true error is commonly expressed as an absolute value and referred to as the absolute error

$$\text{Absolute error} = |\text{accepted} - \text{measured}|$$

$$\text{Relative error} = \frac{|\text{measured} - \text{accepted}|}{\text{accepted value}}$$

$$\text{Percent relative error } (\mathcal{E}_a) = \frac{|\text{measured} - \text{accepted}|}{\text{accepted value}} \times 100\%$$

Error

Determining Error

- Suppose you use a thermometer to measure the boiling point of pure water at standard pressure.
- The thermometer reads 99.1°C .
- You probably know that the true or accepted value of the boiling point of pure water at these conditions is actually 100.0°C .

Error

Determining Error

- For the boiling-point measurement, the error is $99.1^{\circ}\text{C} - 100^{\circ}\text{C}$, or -0.9°C .
- For the boiling-point measurement, the error is $100^{\circ}\text{C} - 99.1^{\circ}\text{C}$, or 0.9°C .

Error

Calculating Percent Error

The boiling point of pure water is measured to be 99.1°C . Calculate the percent error.

Error

Solve for the unknown.

Substitute the equation for error, and then plug in the known values.

$$\text{Percent error } (\mathcal{E}_a) = \frac{|\text{experimental value} - \text{accepted value}|}{\text{accepted value}} \times 100\%$$

$$= \frac{|99.1^\circ\text{C} - 100.0^\circ\text{C}|}{100.0^\circ\text{C}} \times 100\%$$

$$= \frac{0.9^\circ\cancel{\text{C}}}{100.0^\circ\cancel{\text{C}}} \times 100\% = 0.9\%$$

Error

- Often, when performing computations, we may not be concerned with the sign of the error but are interested in whether the absolute value of the percent relative error is lower than a prespecified tolerance ε_s . Therefore, it is often useful to employ the absolute value for expressing

- For such cases, the computation is repeated until

$$\varepsilon_a < \varepsilon_s$$

- This relationship is referred to as a stopping criterion.
- If it is satisfied, our result is assumed to be within the prespecified acceptable level

Error

- For numerical methods, the true value will only be known when we deal with functions that can be **solved analytically**.
- However, in real-world applications, we will obviously not know the true answer a priori.
- For these situations, an alternative is to normalize the error **using the best available estimate of the true value**, that is, to **the approximation itself**, as

$$\varepsilon_a = \frac{\text{Approximate error}}{\text{Approximation}} \times 100\%$$

Error

- One of the challenges of numerical methods is to determine error estimates in the absence of knowledge regarding the true value.
- For example, certain numerical methods use iteration to compute answers.
- In such cases, a present approximation is made based on a previous approximation.
- This process is performed repeatedly, or iteratively, to successively compute (hopefully) better and better approximations.

Error

- For such cases, the error is often estimated as the difference between the previous and present approximations.
- Thus, **the percent relative error** is determined according to;

$$\mathcal{E}_a = \frac{\text{Present Approximation} - \text{Previous Approximation}}{\text{Present Approximation}} \times 100\%$$

- Often, when performing computations, we may not be concerned with the sign of the error but are interested in whether the absolute value of the percent relative error is lower than a prespecified tolerance $\mathcal{E}_s$
- For such cases, the computation is repeated until

$$|\mathcal{E}_a| < \mathcal{E}_s$$

Error Estimation

- In mathematics, functions can often be represented by infinite series. For example, the exponential function can be computed using Maclaurin series expansion

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + \frac{x^n}{n!} \dots (1)$$

- Thus, as more terms are added in sequence, the approximation becomes a better and better estimate of the true value of e^x

Error Estimation

- It can be shown (Scarborough, 1966) that if the following criterion is met, we can be assured that the result is correct to at least n significant figures.

$$\mathcal{E}_s = (0.5 \times 10^{2-n})\%$$

- This can be employed to determine the error criterion that ensures a result that is correct to at least three significant figures

$$\mathcal{E}_s = (0.5 \times 10^{2-3})\% = 0.05\%$$

Error Estimation

- Starting with the simplest version, $e^x = 1$, add terms one at a time in order to estimate $e^{0.5}$.
- The true value for $x=0.5$ is $e^{0.5} = 1.648721 \dots$
- *For $n=1$, $e^{0.5} = 1 + 0.5 = 1.5$*
- This represents a true percent relative error

$$\mathcal{E}_t = \frac{1.648721 - 1.5}{1.648721} \times 100\%$$

$$\mathcal{E}_t = 9.02\%$$

$$\mathcal{E}_a = \frac{\text{Present Approximation} - \text{Previous Approximation}}{\text{Present Approximation}} \times 100\%$$

$$\mathcal{E}_a = ((1.5 - 1.0)/1.5) \times 100\% = 33.3\%$$

Error Estimation

Terms	Result	$\epsilon_t\%$	$\epsilon_a\%$
1. n=0	1	39.3	
2. 1	1.5	9.02	33.3
3. 2	1.625	1.44	7.69
4. 3	1.6458333333	0.175	1.27
5. 4	1.648437500	0.0172	0.158
6. 5	1.648697917	0.00142	0.0158

True value 1.648721

Thus, after six terms are included, the approximate error falls below $\epsilon_s = 0.05\%$, and the computation is terminated.

Computer Application

- To do this for the same problem as above example, the series expansion can be expressed as

$$e^x = \sum_{i=0}^n \frac{x^i}{i!}$$

- `#include <stdio.h>`
- `#include <stdlib.h>`
- `#include <math.h>`
- `double fact(double n) { //Calculating
factorial`
- `double result = 1.0;`
- `for(double i=1;i<=n;i++){`
- `result *= i;`
- `}`
- `return result;`
- `}`

- `int main() {`
- `double np,n,et,es,ea,tv = 1.648721,pv,pri_v,x=0.5; //e power 0.5`
- `printf("Enter the degree of precision : ");`
- `scanf("%lf",&np);`
- `es = (0.5 * pow(10, (2-np)));`
- `printf("Precision %.10f and Stopping criteria = %.10f\n",np,es);`
- `printf("True Result = %g\n\n",tv);`
- `int i=0; ea=1; pri_v=0; //initial estimation and previous value`
- `while(1){`
- `pv += (pow(x,i)/fact(i));`
- `et = (fabs(tv - pv)/tv)*100; //using true value`
- `ea = (fabs(pv - pri_v)/pv)*100; //using present approximation`
- `printf("n=%d,\tResult = %.10f,\tet = %.10f,\tea = %.10f\n",`
- `i,pv,et,ea);`
- `if(et<es) break;`
- `i++; pri_v = pv;`
- `}`
- `return 0;`
- `}`

6 digits

Select E:\Daffodils\Spring_2022\Error_0\bin\Debug\Error_0.exe

Enter the degree of precision : 3

Precision 3.000000 and Stopping criteria = 0.050000

True Result = 1.64872

n=0,	Result = 1.000000,	et = 39.346924,	ea = 100.000000
n=1,	Result = 1.500000,	et = 9.020386,	ea = 33.333333
n=2,	Result = 1.625000,	et = 1.438752,	ea = 7.692308
n=3,	Result = 1.645833,	et = 0.175146,	ea = 1.265823
n=4,	Result = 1.648438,	et = 0.017195,	ea = 0.157978

Process returned 0 (0x0) execution time : 1.896 s

Press any key to continue.

■

10 digits

E:\Daffodils\Spring_2022\Error_0\bin\Debug\Error_0.exe

Enter the degree of precision : 6

Precision 6.0000000000 and Stopping criteria = 0.0000500000

True Result = 1.64872

n=0,	Result = 1.0000000000,	et = 39.3469240702,	ea = 100.0000000000
n=1,	Result = 1.5000000000,	et = 9.0203861054,	ea = 33.3333333333
n=2,	Result = 1.6250000000,	et = 1.4387516141,	ea = 7.6923076923
n=3,	Result = 1.6458333333,	et = 0.1751458656,	ea = 1.2658227848
n=4,	Result = 1.6484375000,	et = 0.0171951470,	ea = 0.1579778831
n=5,	Result = 1.6486979167,	et = 0.0014000752,	ea = 0.0157952930
n=6,	Result = 1.6487196181,	et = 0.0000838192,	ea = 0.0013162571
n=7,	Result = 1.6487211682,	et = 0.0000101991,	ea = 0.0000940183

Process returned 0 (0x0) execution time : 4.803 s

Press any key to continue.

Error

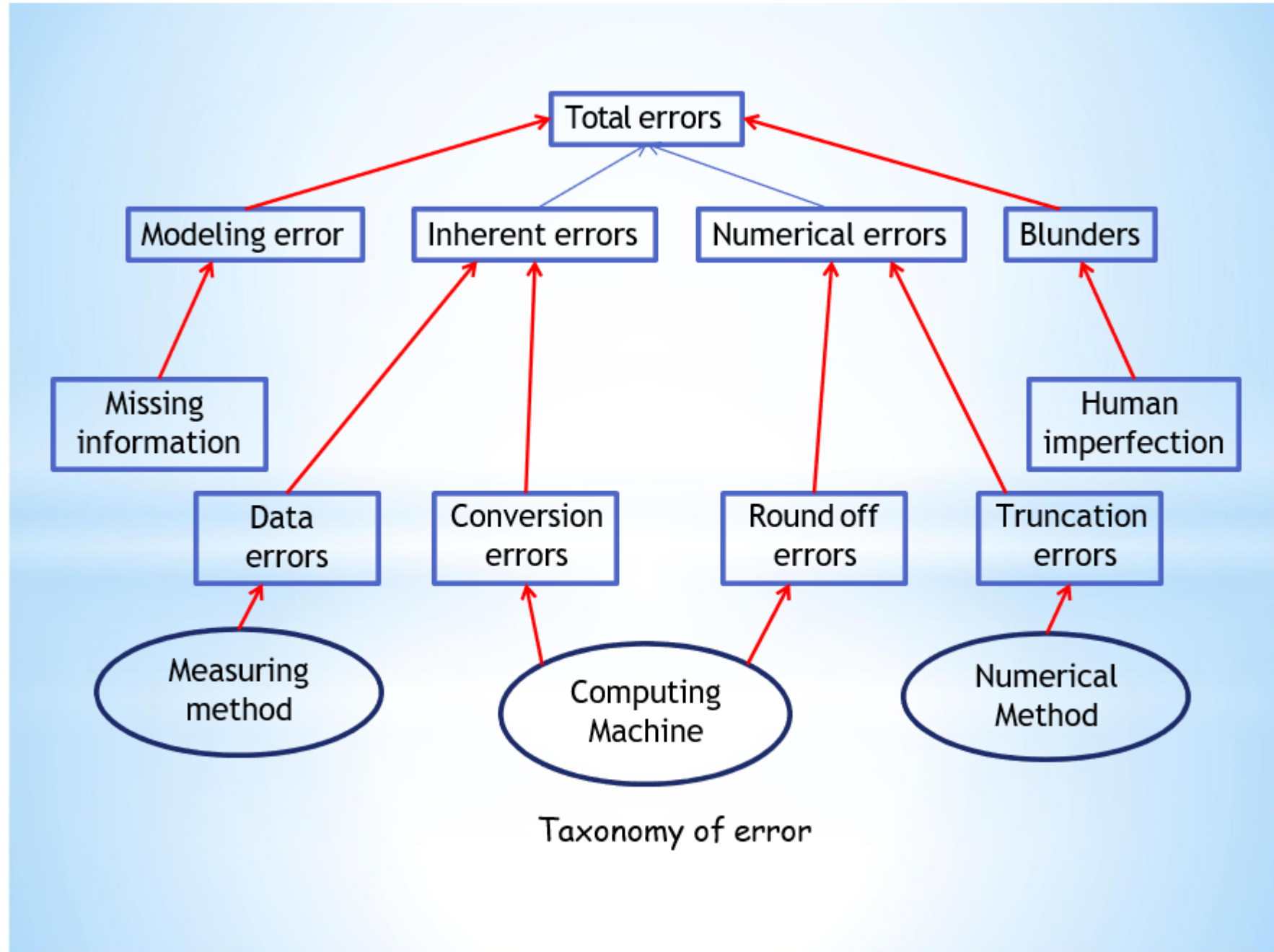
Errors came in a variety of forms and sizes; some are avoidable, some are not.

For example:

- Data conversion and round-off error can not be avoided, but human error can be eliminated.
- Although certain errors can not be eliminated completely, we must at least know the bounds of these errors to make use of our final solution.

It is therefore essential to know how errors arise, how they grow during the numerical process, and how they affect the accuracy of a solution.

Error



Error

Data error: When data for a problem are obtained by some experimental means, and have a limited accuracy and precision. This is due to some limitations of the instruments and reading.

Conversion error: Arises due to the limitation of the computer to store the data exactly.

Round off error: When a fixed no. of digits is used to represent an exact no..

For example,

$$\pi = 3.141592653589793238462643\dots$$

Truncation error: Arises from using an approximation in place of an exact mathematical procedure.

$$e^x = \sum_{i=0}^n \frac{x^n}{n!}$$

Inherent error: Errors that are present in the data supplied to the model.

Numerical error: This error was introduced during the process of implementing the numerical method.

Modeling Error

Mathematical models are the **basis for numerical solutions**.

- **Model formulated** to represent a physical process using certain parameters involved in the situation.
- In many situations, it is impractical or impossible to include all the real problems and, therefore, certain **simplifying assumptions are made**.

For Example:

Developing a model for **calculating the force acting** on a falling body.

- We **may not be able to estimate** the air resistance coefficient (drag coefficient) properly or determine the direction and magnitude of wind force acting on the body, and so on.
- To simplify the model, we may assume that **resistance is linearly proportional** to the velocity of the falling body, or we may assume that there is no wind force acting on the body.

Blunders

- Error that are caused due to human imperfection. Such error may cause a very serious disaster in the result.
- It should be possible to avoid them to a large extent by acquiring a sound knowledge of all aspect of the problem as well as numerical process.
- Human error can occur at any stage of the numerical processing cycle . Some common types of errors are:-
 - Lack of understanding of the problem
 - Wrong assumption
 - Overlooking some basic assumptions required for formulating the model
 - Selecting the wrong numerical method for solving the mathematical model
 - Wrong guessing of the initial value

Numerical Errors (also known as procedural errors)

Introduced during the process of implementation of a numerical method. They come in two forms

Round off

Truncation errors

Total numerical error is the summation of these two errors. The total error can be reduced by devising suitable techniques for implementing the solution

Round off Errors

- When fixed no. of digits are used to represent exact no..
- Since no. are stored at every stage of computation, round off error is introduced at the end of every arithmetic operation.
- Even though an individual round off error could be very small. The cumulative effect of a series of computation can be very significant.

Roundoff Errors

- Roundoff errors arise because digital computers cannot represent some quantities exactly
- There are two major aspects of roundoff errors involved in numerical calculations:
 1. Digital computers have magnitude and precision limits on their ability to represent numbers.
 2. Certain numerical manipulations are highly sensitive to roundoff errors. This can result from both mathematical considerations and the way computers perform arithmetic operations.

Rounding a number can be done in two ways

Chopping

Symmetric rounding

Chopping (Truncation Method):-

Extra digits are chopped. This is called **truncating** the number.

Example:

42.7893 will be stored as 42.78, and the digit 93 will be dropped.

Symmetric Rounding

Last retained significant digit is rounded up by 1 if the first discarded digit is larger than or equal to 5; otherwise, the last retained digit is unchanged.

Example :

42.7893 becomes 42.79

46.5432 becomes 46.54

Computer Number Representation

- Numerical roundoff errors are directly related to the manner in which numbers are stored in a computer.
- A number system is merely a convention for representing quantities.
- The number system that we are most familiar with is the decimal, or base-10.
- Numbers on the computer are represented with a binary, or base-2, system
- $(10101101)_2 = 2^7 + 2^5 + 2^3 + 2^2 + 2^0 = 128 + 32 + 8 + 4 + 1 = (173)_{10}$

Floating Point Number

- Fractional quantities are typically represented in computers using floating-point format. In this approach, **which is very much like scientific notation**, the number is expressed as

$$\pm s \times b^e$$

- where s = the significand (or mantissa), b = the base of the number system being used, and e = the exponent.
- For example, a value like 0.005678 could be represented in a wasteful manner as 0.005678×10^0
- However, normalization would yield 5.678×10^{-3} , which eliminates the useless zeroes

Implications of Floating-Point Representation

- Let a base-10 computer have a 5-digit word size. Assume one digit is used for the sign, two for the exponent, and two for the mantissa.
- For simplicity, assume that one of the exponent digits is used for its sign, leaving a single digit for its magnitude
- A general representation of the number following normalization would be

$$s_1 s_0 d_2 d_1 d_0 \quad \rightarrow \quad s_1 d_1 . d_2 \quad \times 10^{s_0 d_0}$$

where s_0 and s_1 = the signs,

d_0 = the magnitude of the exponent,

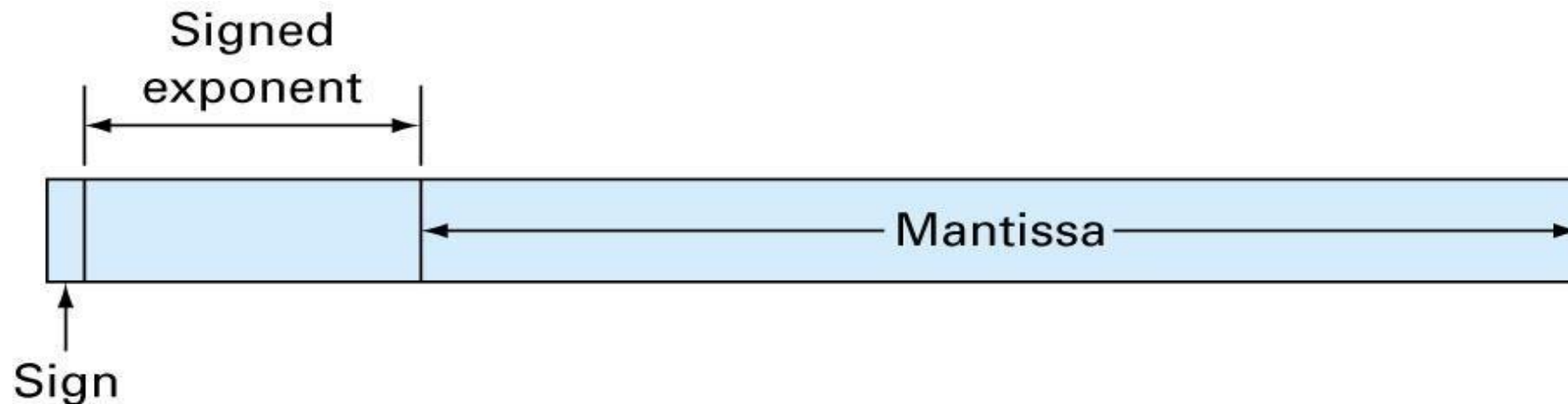
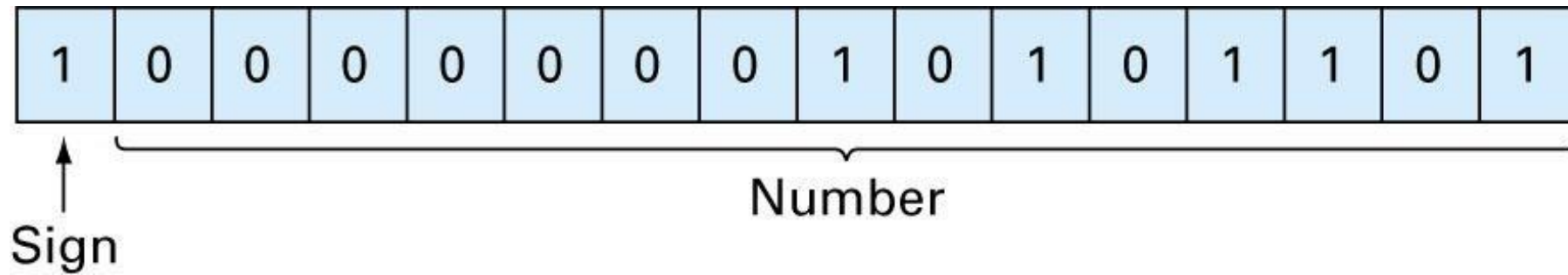
and d_1 and d_2 = the magnitude of the significand digits.

Implications of Floating-Point Representation

- Largest possible value in base-10, that is, 9:
- Largest value = $\pm 9.9 \times 10^{+9}$
- Smallest value = $\pm 1.0 \times 10^{-9}$
- We could not use it to represent a quantity like Planck's constant
 $6.626 \times 10^{-34} \text{ J}\cdot\text{s}$
- If we increase the mantissa by one digit, the maximum/minimum value increases slightly to
 $\pm 9.99 \times 10^{\pm 9}$
- In contrast, a one-digit increase in the exponent raises the maximum/minimum by 90 orders of magnitude to
 $\pm 9.9 \times 10^{\pm 99}$

Computer Number Representation

The representation of the decimal integer (-173) on a 16-bit computer using the signed magnitude method.



IEEE-754 Floating-Point Format

Most modern computers follow the **IEEE 754 standard** for floating-point numbers.

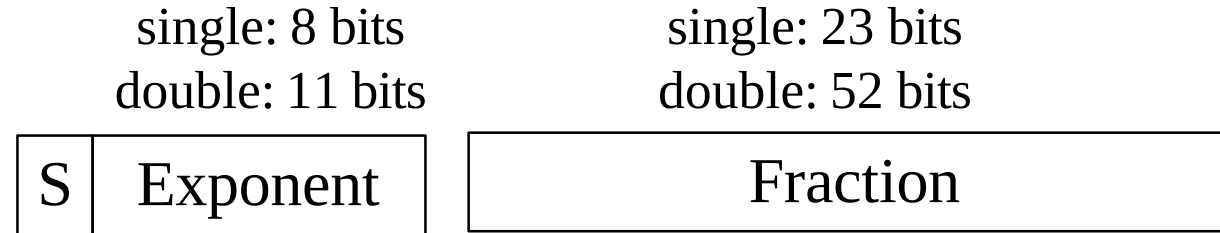
It defines two commonly used formats:

- **Single Precision (32-bit)**
- **Double Precision (64-bit)**

Both formats divide the binary representation into **three fields**:

Field	Purpose
Sign bit	Indicates positive (0) or negative (1)
Exponent	Stores the power of 2 (with bias)
Mantissa (Fraction)	Stores significant digits (precision)

IEEE-754 Floating-Point Format



$$x = (-1)^s \times (1 + \textit{Fraction}) \times 2^{(\textit{Exponent} - \textit{Bias})}$$

- S: sign bit (0 = non-negative, 1 = negative)
- **Normalize significand: $1.0 \leq |\textit{significand}| < 2.0$**
 - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
 - **Significand is a Fraction with the “1.(Fraction)” restored**
- **Exponent = Actual Exponent + Bias (or, Actual Exponent = Exponent – Bias)**
- Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1023

Single-Precision Range

- Exponents **00000000=0** and **11111111=255** reserved
- **Smallest value**
 - Exponent: 00000001=1
Actual Exponent = $1 - 127 = -126$
 - Fraction/Mantissa: **1.000...00** = significand = 1.0
 - **$\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$**
- **Largest value**
 - Exponent: 11111110=254
Actual Exponent = $254 - 127 = +127$
 - Fraction: **1.111...11 (23 ones)** = significand ≈ 2.0
 - **$\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$**

Double-Precision Range

- Exponents 0000...00 = 0 and 1111...11 = 2047 reserved
- Smallest value
 - Exponent: 000000000001=1
Actual Exponent = $1 - 1023 = -1022$
 - Fraction: 1.000...00 = significand = 1.0
 - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value
 - Exponent: 111111111110=2046
Actual Exponent = $2046 - 1023 = +1023$
 - Fraction: 1.111...11 (52 Ones) = significand ≈ 2.0
 - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Floating-Point Example

- Represent -0.625
 - $-0.625 = -0.101 = (-1)^1 \times 1.01_2 \times 2^{-1}$
 - $S = 1$
 - Fraction = $01000\dots00_2$
 - Exponent = $-1 + \text{Bias}$
 - Single: $-1 + 127 = 126 = 01111110_2$
 - Double: $-1 + 1023 = 1022 = 01111111110_2$
- Single: $101111111001000\dots00$
- Double: $101111111111001000\dots00$

Floating-Point Example

- Represent -13.25 in Single & Double Precision
- $-13.25 = -1101.01 = (-1)^1 \times 1.10101_2 \times 2^3$
- $S = 1$
- Exponent = 3 + Bias
 - Single: $3 + 127 = 130 = 10000010_2$
 - Double: $3 + 1023 = 1026 = 10000000010_2$
 - Mantissa = 101010000000000000000000 (drop leading 1)
- Single: $110000010101010...00$
- Double: $1100000000101010100...00$

Floating Point Arithmetic in Computers

- When two floating-point numbers are added, the numbers are first expressed so that they have the same exponents.
- For example, if we want to add $1.557 + 0.04341$, the computer would express the numbers as

$$0.1557 \times 10^1 + 0.004341 \times 10^1.$$

- Then the mantissas are added to give $0.160041 \times 10^1 = 1.60041$.
- More examples,
- Suppose that we are subtracting 26.86 from 36.41. That is,

$$0.3641 \times 10^2 - 0.2686 \times 10^2 = 0.0955 \times 10^2 = 9.5500$$

[Notice that the zero added to the end of the mantissa is not significant but is merely appended to fill the empty space created by the shift.]

Floating Point Arithmetic in Computers

- Even more dramatic results would be obtained when the numbers are very close, as in $0.7642 \times 10^3 - 0.7641 \times 10^3 = 0.0001 \times 10^3$
which would be converted to $0.1000 \times 10^0 = 0.1000$.
- Thus, for this case, three nonsignificant zeros are appended.
- The subtraction of two nearly equal numbers is called **subtractive cancellation**.

Floating Point Arithmetic in Computers

Subtractive cancellation

- Since computers have finite precision, the leading digits cancel out, leaving only the difference in the less significant digits. This results in loss of significant digits and large relative error, even if the inputs are accurate.
- 4-digit arithmetic:
 - $x=9.876$ (stored as 9.876)
 - $y=9.873$ (stored as 9.873)
 - Subtract: $9.876-9.873=0.003$
 - Now consider if $x=9.876$ and $y=9.875$ (close numbers).
 - True result: $x-y=0.001$
 - But in finite precision (say only 3 significant digits stored):
 - $x=9.88$ and $y=9.88 \rightarrow$ subtraction gives 0.00 (loss of accuracy).

Floating Point Arithmetic in Computers

Adding a Large and a Small Number

Suppose we add a small number, 0.0010, to a large number, 4000, using a hypothetical computer **with the 4-digit mantissa and the 1-digit exponent**. We modify the smaller number so that its exponent matches the larger,

$$\begin{array}{r} 0.4000 \quad \cdot 10^4 \\ 0.0000001 \cdot 10^4 \\ \hline 0.4000001 \cdot 10^4 \end{array}$$

which is chopped to $0.4000 \cdot 10^4$. Thus, we might as well not have performed the addition!

This type of error can occur in the computation **of an infinite series**. The **initial terms** in such series **are often relatively large in comparison with the later terms**. Thus, after a few terms have been added, we are in the situation of adding a small quantity to a large quantity.

One way to mitigate this type of error is to sum the series in reverse order—that is, in ascending rather than descending order.

Floating Point Arithmetic in Computers

- **Large Computations:** Certain methods require extremely large numbers of arithmetic manipulations to arrive at their final results. **These computations are often interdependent.** That is, the later calculations are dependent on the results of earlier ones.
- Consequently, even though **an individual round-off error could be small**, the cumulative effect over the course of a large computation can be significant.

Minimizing Round-off Error

The effects of round-off error can be minimized by changing the computational algorithm, although it must be devised on a case-by-case basis. Some useful strategies include:

- ✓ **Double precision**
- ✓ **Grouping**
- ✓ **Taylor expansions**
- **Rewriting the equation to avoid subtraction**

Minimizing Round-off Error

- **Double Precision**

In a double precision, 8 bytes, or equivalently 64 bits, are used to store one real number. In this format 1 bit is used for sign, 11 bits are used for exponent, and 48 bits are used for mantissa.

- **Grouping**

When the small numbers are computed, e.g. addition, subtraction, etc., grouping them helps to reduce round-off errors. For example, to add 0.00001 to unity ten thousand times one can grouped into 100 groups and each group consists of 100 small values.

Minimizing Round-off Error

- **Taylor Expansions**

As α approaches 0, accuracy of a **numerical** evaluation for

$$f(\alpha) = \frac{\sin(1+\alpha) - \sin(1)}{\alpha}$$

becomes **very poor** because round-off errors. By using Taylor expansion, we can rewrite the equation so that the accuracy for α is **improved** as

$$f(\alpha) \cong \cos(1) - 0.5\alpha^2 \sin(1)$$

Minimizing Round-off Error

- **Rewriting the equation to avoid subtraction**

consider the equation of

$$f(x) = x(\sqrt{x+1} - \sqrt{x})$$

For an increasing of values x , the calculation of the equation above has a **loss-of-significance error**. To avoid this error, one can reformulate it to get. Multiplying $f(x)$ by $\frac{\sqrt{x+1} + \sqrt{x}}{\sqrt{x+1} + \sqrt{x}}$ becomes

$$f(x) = \frac{x}{\sqrt{x+1} + \sqrt{x}}$$

Truncation Error

- Numerical solutions are mostly approximations for the exact solution
- Most numerical methods are based on approximating a function by polynomials
- How accurately does the polynomial approximate the true function?
- Comparing the polynomial to the exact solution, it becomes possible to evaluate the error, called the *truncation error*

Truncation Error

- Truncation errors are those that result from using an approximation in place of an exact mathematical procedure.
- Example 1: approximation to a derivative using a finite-difference equation:

$$\frac{dv}{dt} \approx \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

Truncation Error

Example 2: **The Taylor Series**

- The Taylor theorem states that **any smooth function can be approximated as a polynomial.**
- The Taylor series provides a means to express this idea mathematically.

$$f(x) = f(x_0) + \frac{x - x_0}{1!} f'(x_0) + \frac{(x - x_0)^2}{2!} f''(x_0) + \cdots + \frac{(x - x_0)^n}{n!} f^{(n)}(x_0) + R_n$$